Project Report

Author:

Name: Abhrojyoti Ghosh, Student ID: DS23f1002104, email id: 23f1002104@ds.study.iitm.ac.in. I am currently a student at the Diploma level. I am also a 4th-year BS-MS Dual Degree student at IISER Kolkata.

Description:

The goal of this project is to build a 4-wheeler vehicle parking application. We are tasked with implementing Role-Based Authentication for Users and Admins, allowing admins to perform CRUD operations on Parking Lots.

Technologies Used:

The entire app is built based on Flask and its extensions:

- Flask-WTF: To secure form validation and CSRF Protection,
- Flask-Login: To have a login functionality and role based access,
- Flask-SQLAlchemy: To create a database in the application
- To implement search functionality in my application, rapidfuzz is used.

DB Schema Design:

The DB Schema closely follows the one given in the project statement.

User

Description: Stores information about users (both regular and admin).

Columns:

1. id: String(36), Primary Key, UUID. Unique identifier for each user.
2. username: String(36), Unique, Not Null. Username of the user.
3. email: String(120), Unique, Not Null. Email address of the user.
4. full_name: String(80). Full name (optional).
5. password: String(256), Not Null. Hashed password.
6. is_admin: Boolean, Default False. Marks if the user is an admin.

Relationships: One-to-many with Reservation (user can make multiple reservations)

ParkingLot

Description: Contains information about individual parking lots.

Columns:

1. id: String(22), Primary Key, shortuuid. Unique ID for the parking lot.
2. prime_location_name: String(100), Not Null. The prime location of lot
3. price_per_hour: Float, Not Null. Price charged per hour.

4. address: String(255), Not Null. Address of the parking lot.
5. pin_code: String(10), Not Null. Postal code.
6. max_spots: Integer, Not Null. Maximum number of parking spots.

Relationships:

1. One-to-many with ParkingSpot
2. One-to-many with Reservation

ParkingSpot

Description: Represents a specific spot in a parking lot.

Columns:

1. id: Integer, Part of Primary Key. Spot number within the lot.
2. lot_id: String(22), Foreign Key to ParkingLot.id, Not Null. References to the parent lot.
3. status: String(1), Default 'A' ;{'A' = Available, 'O' = Occupied}

Constraints: Composite Primary Key: (id, lot_id)

Relationships:

1. Many-to-one with ParkingLot
2. One-to-many with Reservation

Reservation

Description: User reservations of specific parking spots.

Columns:

1. id: String(22), Primary Key, shortuuid.Unique ID of the reservation.
2. status: String(1), Default 'R' {'R' = Reserved, 'F' = Finished}
3. lot_id: String(22), Foreign Key to ParkingLot.id, Not Null
4. spot_id: Integer, Not Null
5. user_id: String(36), Foreign Key to User.id, Not Null
6. car_id: String(10), Not Null. Vehicle registration number.
7. parking_timestamp: DateTime, Default = now. When the vehicle entered.
8. leaving_timestamp: DateTime, Nullable. When the vehicle left.

Constraints: Composite Foreign Key: (lot_id, spot_id) → ParkingSpot(lot_id, id)

Relationships:

1. Many-to-one with User
2. Many-to-one with ParkingLot
3. Many-to-one with ParkingSpot

Design Idea:

- UUIDs and short UUIDs are used for unique identifiers.
- Composite keys ensure a spot is unique only within a lot, since spot numbers are incremented from 1 to max_spots in each lot.
- Cascade delete removes child records like reservations when a parent (lot/user/spot) is removed.
- Single-character status fields are used for compactness.

Architecture:

1. The main application file, app.py, lies inside the project_root_folder.
2. The controllers are kept all in the Controllers folder; however, in separate files based on their usage and functionalities. Keeping the same in mind, four flask Blueprints were made and each lies in a separate file, namely main.py, auth.py, admin.py and user.py, where each file handles the functionality as their name suggests.
3. To serve the same purpose with modularity and least confusion, the templates are also arranged in four different folders. Since no styling other than Bootstrap 5 is used, the src folder remains empty.
4. The Models folder contains the database and required models in our application.
5. A special file, create_admin.py, is also present in the root folder that helps to auto create an admin when the app is run for the first time. The README.md file in the root folder contains a brief introduction to my app along with instructions on how to run the application. The file requirements.txt captures all the dependencies of the application. In places, it also mentions why they are required when not understood trivially.

Features:

Based on the role of the user (super user or regular user), they can perform different tasks in the app. However, they all share the same login process, and they all can edit their profile.
An admin is allowed to create a lot and auto-create parking spots along with it; no manual spot creation option is given to them. They can see all the lots with available spots. For the unavailable spots, an admin can see who has occupied that spot. An admin can also see the list of all users on the app. An admin can delete a lot if any of its spots are occupied, otherwise he can delete a particular spot, if that is again, unoccupied.
A user, on the other hand, can see which lot has available spots, can book a spot for parking. Can not fill in spot details manually, which shall be allocated randomly. They can see all their reservations and the total time they have spent in a particular parking lot in their summary section.

Declaration:

To build this application about 20-30% help is taken from AI-LLMs. In particular to implement search functionalities, to implement FlaskForms, to design and load the database effectively and to debug AI is been used. However all the backend logics are done by myself and all the help that were taken I understand them properly.

Demo Video:

The demo video of the app can be found Avro's 1.mp4